THE UNIVERSITY OF BRITISH COLUMBIA
Electrical and Computer Engineering Department

CPEN 442 **Introduction to Cybersecurity** **Winter Term 1, 2026**

Assignment 3: Authentication and Hashing

Due date: **Friday, Oct 23, 2026, 11:59 pm**
Hard deadline: **Sunday, Oct 25, 2026, 11:59 pm**

Use Piazza for questions. Instructor and TA support ends at 2 PM on the due date. A 48-hour automatic extension is provided for upload issues. There is no penalty until the hard deadline; no submissions accepted after that date. For long-term issues, contact the instructor *before the due date*.

Total marks: 200 pts

Doing the Assignment: This assignment must be completed individually or in groups of two. You may discuss it with others, but you must not *copy* or share solutions (this includes not having a public repository with your solutions for the duration of the course).

Generative AI Disclosure. You may use generative AI for any part of this assignment and will not be penalized for doing so. However, use it as a tool rather than a substitute for understanding. If you use generative AI, include a final section titled "Generative AI Disclosure" describing how and where it was used. This section must be *between half a page and one page long*, and cannot be written by AI. Please also comment on how helpful the AI was, as the instructor is interested in learning about this.

What to submit

- The file `a3.py`, for which you can find a template with the assignment files. Please do not include any non-default modules, so that we can run your code. This answers questions Q3, Q6, and Q9 of this assignment.
- A pdf file called `a3.pdf` with the answers to questions Q1, Q2, Q4, Q5, Q7, and Q8, and AI disclosure (if applicable). You can create this with Microsoft Word, Google Docs, a text editor, or latex. Just make sure it is clear where each answer is, and that the pdf is readable. You can find a latex template in the assignment page.

1 Offline Dictionary Attacks

In a dictionary attack, the adversary tries to guess user passwords by trying with common passwords from a list. This list is called the *dictionary*. For this question, you can take the dictionary from this link. In an *offline* dictionary attack, the adversary has access to some password digests, and computes hashes locally to guess passwords.

First, let's learn how to compute a password digest with Python. For this assignment, you can use the following packages: `Crypto` (PyCryptodome), `bcrypt` (link), and `hashlib` (link). These are the packages that we have installed when grading the assignment; please avoid other packages as they might raise an error when we grade.

Do your own investigation to learn how to compute a SHA-256 digest using any of those packages (perhaps this is easier with `hashlib`). Here you have a sample string with its corresponding SHA-256 digest in hexadecimal. Check that you get the same digest.

```
x="This is a sample string"
x_binary=x.encode('ascii')
... # Compute the SHA-256 hash and represent it in hexadecimal
sha256_hex = "06e40bc8c495970478e6f742392d718713c06200bd77ead0468b444bf0ba68ea"
```

[10 pts] Question 1: Consider the following password digests:

```
# Digests for Question 1
8a81e9a2dbdc9ccf4ac5091a8a3458b529de766c3320138cf1f9708e537afb01
da938a3a3165693b72dddaa29453919ec9e82803c2510af6dc54efa5
a5b23f338648cb6f21e2cc1284e4d1b7b930ac97
```

These digests have been generated with different popular hash functions, all from the Secure Hashing Algorithm (SHA) family. You can search the Internet to learn more information about the SHA family. All the passwords that generated these digests come from the dictionary from the link above. Using your own code, guess the passwords that generated these digests. Then, in `a3.pdf`, write the answers to these questions:

a) Which hashing algorithm has been used to generate each hash and how did you deduce this?

b) What are the passwords that generated these digests?

Using a secure hash function from the SHA family (e.g., SHA-256) for password storage is not recommended. This is because these hash functions are designed to be secure (e.g., hard

to find pre-images doing anything other than brute-forcing), but are very fast to compute (which allows faster brute-forcing). There are hash functions that are designed to be slow to compute. bcrypt is one of the most popular ones.[1]

In the questions below, you will have to work with digests that are represented using the Modular Crypt Format. You can find more about this format in the following link: `https://passlib.readthedocs.io/en/stable/modular_crypt_format.html`. This format indicates the hashing algorithm, the digest, and other parameters (in the case of bcrypt: the salt and the number of rounds of the hashing algorithm).

[10 pts] Question 2: Consider the following password digests in Modular Crypt Format:

```
# Digests for this question
$2b$06$AS2K2H8nSwdvs2yNoDgtbu8gq5AuErurrhuqd5QJ2/2KlLhiI6D76
$2b$08$Shq4mPuANPJbdqozQkB1NOLhfF1sTX56CIIb68zb7Wh2YhOjPWKWK
$2b$06$dHO22WXdr4/w6PiDHhVYA.MPQ2Oh00ZmwlzTHhQ1Aich/zzcKpiVC
```

Answer the following questions in `a3.pdf`, explaining your answer briefly (one sentence is enough):

- Which hash function has been used to generate the three digests?
- Do the digests have the same salt, or different salts?
- Which of the digests do you think took longer to compute and why?

[40 pts] Question 3: Next, you must program an offline dictionary attack against bcrypt hashes given in Modular Crypt Format. The attack must be implemented as the function `offline_dictionary_attack` in `a3.py`.

```
def offline_dictionary_attack(dictionary: list[str], digests: list[str]) ->
    list[str]:
    """This attack receives a list of strings as dictionary of possible
       passwords, and another list of strings as bcrypt digests in modular
       crypt format.
    It returns the list of passwords that generated those digests, in the same
       order."""

    pwd_list = [''] * len(digests) # Replace with the actual attack
    return pwd_list
```

We will only test this function with digests produced by the bcrypt algorithm. You can also assume the salt will be 22 characters. You can find some examples of digest files similar to the

[1] This assignment was created with `bcrypt` version `5.0.0`

ones that will be used to grade the assignment, and the expected solutions, in the assignment files (`q3_sample<i>.txt` and `q3_sample<i>_sol.txt`). You can run `q3_test.py` with your `a3.py` solution and check if your attack is working correctly. To get maximum points, your function must be smart and try to *minimize the number of hash calls*.

[15 pts] Question 4: Take a look at the files `q3_sample1.txt`, `q3_sample2.txt`, and `q3_sample3.txt`. Explain in `a3.pdf` which of these files is easier to break ("breaking" here means guessing all the passwords that were used to generated those digests) and which is the hardest. You can run `q3_test.py` to see how long your attack takes, but you can also answer this question using your intuition.[2]

[2]This means that, if you get a result that you think it makes no sense, maybe your previous question is wrong?

2 Offline Brute-Force Attacks

We just saw that dictionary attacks are feasible, even when someone uses a recommended hash function for password storage, such as bcrypt. Brute forcing is harder: now the space of possible passwords is way bigger than just a dictionary. Let's assume that, in our authentication system, passwords can contain any ASCII printable character. In Python, you can run this to see these characters:

```
import string
ascii_printable = string.digits + string.ascii_letters + string.punctuation
print(ascii_printable)
print(len(ascii_printable)) # This is the number of possible characters
```

Take file `q5_test.py` and run it on your computer. This file provides estimates for your computer's speed at computing bcrypt hashes with a certain number of rounds, as well as SHA-256 hashes. Make sure you understand this code. You can read more about Python's `timeit` module here.

[10 pts] Question 5: Run the script on your computer, adjusting the `hash_number` variable if your computer is taking too long. This should give you an estimate of the hashing speed of your computer. Then, compute what is the *maximum password length* such that your computer would be able to compute the digests of all possible passwords of that length in *under one day*. Answer this question, explaining how you got to the final number, in `a3.pdf` for both bcrypt (with 4 rounds) and SHA-256.

Hopefully, this convinces you about the advantages of using bcrypt vs. SHA-256 for password storage. Note that, if you run bcrypt with a higher number of rounds for a single password, your computer will do fine. This means the authentication server is OK with using a larger number of rounds, but this really *deters* a password guessing attack.

3 Online attacks

In online attacks, the attacker has to interact with an authentication server to check whether a possible password is correct. This makes guessing the password way harder. In this assignment, we will see that *side-channel* leakage can make password guessing attacks faster. In particular, we will consider a *timing side-channel attack*: this attack exploits the time it takes for the server to run a program.

Consider a server that stores the users' passwords in plaintext (which we know is not great) and, upon receiving an authentication attempt, just checks if the submitted password is correct:

```
...
if received_pwd == stored_pwd:
    # Authentication succeeded
    ...
else:
    # Authentiction failed
    return
```

At a logical level, this seems fine: we are literally just checking if the password the user has provided matches the one stored in the server's dataset. However, Python performs string comparisons character-by-character sequentially, meaning that evaluating the two statements below take a different amount of time! (on average).

```
"password123" == "abcabcabc" # This is faster
"password123" == "password124" # This is slower
```

If you are curious, you can use `timeit` to check this by yourself. This is an important lesson: sometimes, a program or system that looks secure might be leaking information through unexpected means that an attacker might be able to exploit.

For this assignment, we will do something less fancy, but easier to grade consistently. We will consider a server that literally performs the password comparison character-by-character and, on top of that, waits a bit after checking each character, to amplify this timing leakage. This assignment has been tricky in the past, because an attack can work when a processor is free, but fail when it is busy running other programs. This year, instead of having the processor sleep for milliseconds after checking a character, we will instead "simulate time" in a controlled way. This is not "realistic", but this way we can make sure your grade does not depend on how busy our servers are when grading.

Consider the Python script `q6_server.py`, which simulates an authentication server. This server returns either `True` (authentication succeeded) or `False` (authentication failed), as

well as the "virtual time" it took to run the password check. Using this virtual time, you must program an attack that calls `check_password` repeteadly until it guesses the user's password.

The server initialization looks like this:

```
def __init__(self, seed, L):
    """This simulates the user's registration in the server with a certain
       password, stored in self.password"""
    random.seed(seed)
    self.password = ''.join(random.choices(VALID_CHARS, k=L))
    self._calls = 0 # Do not access this variable from the outside!
```

This code simply chooses a random password of length `L`. Do not touch the variable `_calls`; we will use this to check how many times you have called the password checker.

The server exposes the following function:

```
def check_password(self, password: str) -> tuple[bool, int]:
    """This simulates the server checking if the provided password matches the
       one stored in self.password"""
    self._calls += 1
    virtual_time = 0
    if len(password) != len(self.password):
        return False, virtual_time
    for i, char in enumerate(password):
        virtual_time += self._random_delay()
        if char != self.password[i]:
            return False, virtual_time
    return True, virtual_time
```

This function first checks that the length of the password provided matches the one stored in `self.password`, and then proceeds to check, character by character, that the passwords are identical. The function keeps track of some `virtual_time`, which is meant to simulate natural delays that might happen while running code. The function `_random_delay()` can add delay equal to 1, 2, 3, or 4 time units. The password checker returns a boolean indicating whether the provided password was correct or not, and the virtual time. Your attack can use this virtual time to guide the password guessing process.

[45 pts] Question 6: Provide your implementation of `online_attack` in `a3.py`. You can assume the password is between 5 and 10 characters long, from the set of ASCII printable characters as seen above. You can use `q6_test.py` to check how many calls to the password checker it takes you to crack the password. The function `plot_performance()` shows the performance of your attack: aim to have a median time (the center line of the box) below the red line for maximum points.

4 Two-factor Authentication

The following link contains a list of services that support 2-factor authentication: `https://2fa.directory/` Go to the link and pick a service from that list that you use.

[10 pts] Question 7: Answer, in `a3.pdf`: what service did you pick, and what forms of two-factor authentication does it support? What would it take for an attacker to circumvent that service's two-factor authentication? (Give an example in which the adversary would bypass the authentication. Simply think of what could realistically happen that would allow someone to authenticate as you in that service.)

5 Authentication in UBC

UBC uses the Campus Wide Login (CWL) system to provide access to Canvas, Workday, etc. On the UBC website, you can find a list of CWL FAQs that explain the password policy (click here). Recall what we have seen in the classroom about password hygiene and the NIST Password Guidelines, published as NIST Special Publication 800-63B-4 (click here).

[15 pts] Question 8: Read the answer to the question "Why do I have to change my CWL password?" on the website, as well as "What are the CWL password requirements?". Briefly comment on the answers that the UBC website gives to those questions (e.g., are they good decisions from a security perspective based on what you have learned in this course?). For each of the two UBC answers, find the specific recommendation in the NIST Password Guidelines that applies to it, quote it, and give the number of the section where you found it. Write your answers in `a3.pdf`.

6 Encrypting a hash to provide authentication

In the classroom, we saw that using a hash for authentication/integrity is only useful if we can transmit that hash through a secure channel. Consider the following (simplified) authentication server. This server can generate tokens for a username and password. The server generates the plaintext token bytes M and then computes $E_K(M \parallel h(M))$ with a secret key K. This is the implementation, which you can find in `q9_server.py`:

```python
def generate_guest_token(self, name: str, pwd_b64: str) -> tuple[bytes, bytes]:
    """Receives name and password strings, and returns an encrypted user
       token."""
    # No input sanitization (allows &)
    token_bytes = b'name=' + name.encode('ascii')
    token_bytes += b'&pwd=' + base64.b64decode(pwd_b64.encode('ascii'))
    token_bytes += b'&role=guest'
    token_digest = SHA256.new(data=token_bytes).digest()
    ciphertext, nonce_or_iv = self._encrypt(token_bytes + token_digest)
    return ciphertext, nonce_or_iv
```

Note that

1. The password is provided as a base64 string.
2. The server builds a binary token with the name, the decoded password bytes, and the guest role.
3. Then, the server computes a SHA-256 digest, and encrypts the token concatenated with its digest.

When parsing a received token in `read_token`, the server first decrypts and then verifies the hash. Intuitively, this prevents an attacker from modifying the ciphertext. However... this does not work. In particular, if the encryption is AES in CBC mode, an attacker can forge an `admin` or `super_admin` token.

[45 pts] Question 9: Program the function `forge_admin_token_cbc_hash`, which receives the server's call to generate a guest token (using AES-CBC), and returns a modified token, IV, username, and password, such that the server correctly reads the token, but the new role is:

- "`admin`": 30 pts.
- "`super_admin`": 45 pts.

The grading of this question will follow `q9_test`.

Run `q9_test.py` to test your solution. Make sure you understand the solution, and good luck!